Learning Objectives

- Understand how a stack can be implemented using an array and how `push` and `pop` manage its contents.
- Understand that variables and array elements occupy locations in memory and that these locations have addresses.
- Understand how pointers can be used to refer to and access values stored in memory.
- Understand pointer arithmetic and explore its relationship to arrays and array indexing.

Step 1 — Building a Stack with an Array

In the lecture videos, you learned how arrays can be used to store several values of the same type. We will now use an array to implement a simple stack that can store up to five integer values.

A stack provides two basic operations:

- `push` adds a value to the top of the stack.
- `pop` removes the value at the top of the stack and returns it.

The variable `stackSize` keeps track of how many elements are currently stored on the stack. Study the implementation in `stack.abc` below before continuing. Try to understand what happens to `stack` and `stackSize` when `push()` or `pop()` is called.

```
@ <stdio.hdr>

global
    stack: array[5] of int,
    stackSize: int = 0;

fn push(val: int)
{
    assert(stackSize < sizeof(stack) / sizeof(stack[0]));
    stack[stackSize++] = val;
}

fn pop(): int
{
    assert(stackSize > 0);
    return stack[--stackSize];
}

fn main()
{
    push(3);
    push(4);
    pop();
}
```

Step 2 — Accessing the Top of the Stack

So far, `pop()` is our only operation for accessing the value at the top of the stack. However, `pop()` also removes this value from the stack. Implement a function

```
fn top(): int
```

that returns the value at the top of the stack **without removing it**. As with `pop()`, calling `top()` on an empty stack is not allowed. Use `assert()` to detect this case.

Test your implementation with the following main function:

```
fn main()
{
    push(3);
    printf("top = %d\n", top());
    push(4);
    printf("top = %d\n", top());
    pop();
    printf("top = %d\n", top());
}
```

© Michael C. Lehn | CC BY-SA 4.0

This program should produce the following output:

```
lehn$ ./a.out
top = 3
top = 4
top = 3
```

Compare your implementations of `top()` and `pop()`. What is the essential difference between the two operations? In particular, what happens to `stackSize`?

Step 3 — Inspecting the Stack

So far, we have only looked at the value at the top of the stack. To inspect all values that are currently stored on the stack, add the following function:

```
fn info()
{
    printf("stack: ");
    for (local i: int = 0; i < stackSize; ++i) {
        printf("[%d] ", stack[i]);
    }
    printf("\n");
}
```

Modify `main` so that after each `push` or `pop` operation, both the value at the top of the stack and the complete stack contents are displayed. For example, the operations

```
push(3);
push(4);
pop();
```

should produce output similar to:

```
lehn$ ./a.out
top = 3
stack: [3]
top = 4
stack: [3] [4]
top = 3
stack: [3]
```

Step 4 — What Does pop() Actually Do?

After `push(3)`, `push(4)`, and `pop()`, the `info()` function displays

```
stack: [3]
```

But what actually happened to the value `4`?

Remember that our stack is implemented using an array with five elements. The current implementation of `info()` only displays those array elements that are currently part of the stack.

Extend `info()` so that it also displays the remaining elements of the underlying array. Separate the elements that currently belong to the stack from the remaining elements of the underlying array by a vertical bar `|`. Your implementation of `info()` should not depend on the array having exactly five elements. If the size of `stack` is changed, for example from 5 to 10, `info()` should continue to work without any other changes.

Your implementation of `info()` should not depend on the array having exactly five elements. If the size of stack is changed, for example from 5 to 10, `info()` should continue to work without any other changes.

```
MCL:tmp lehn$ ./a.out
top = 3
stack: [3] | [0] [0] [0] [0]
top = 4
stack: [3] [4] | [0] [0] [0]
top = 3
stack: [3] | [4] [0] [0] [0]
```

Look carefully at the last line. What happened to the value `4` when `pop()` was called? Compare this observation with your answer about `stackSize` in Step 2. What will happen to the value `4` when another value is pushed onto the stack?

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — Using Our Stack for Function Calls

In Worksheet 3, we saw that each function call gets its own stack frame containing the data associated with that call. In particular, recursive calls require several stack frames to exist at the same time.

We will now use the simple stack implemented in the previous steps to simulate one small part of this behavior: storing the function parameter `n`.

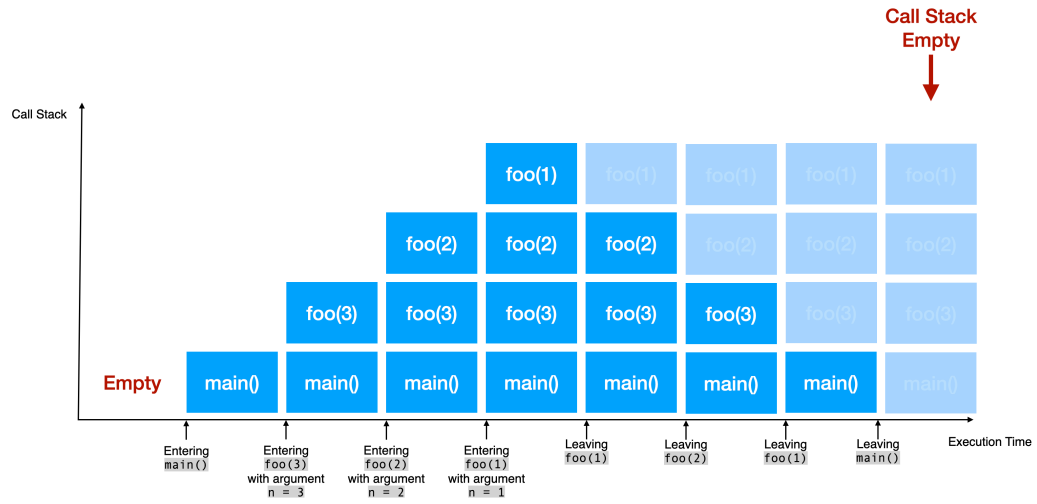
Consider the following recursive program:

```
@ <stdio.hdr>

fn foo(n: int)
{
    printf("entered foo with n = %d\n", n);
    if (n > 1) {
        foo(n - 1);
    }
    printf("leaving foo with n = %d\n", n);
}

fn main()
{
    foo(3);
}
```

The following figure from Worksheet 3 illustrates the stack frames that exist during the execution of this program.



Task

We will now simulate this behavior using the stack that we implemented in the previous steps. Modify the program according to the following rules:

- Replace a call such as foo(3) by

  ```
  push(3);
  foo();
  ```

  The argument is therefore placed on the stack before the function is called.
- Change `foo` so that it no longer has a parameter. Whenever the function needs the value of `n`, obtain it from the top of the stack using `top()`.
- Before `foo()` returns, remove the value belonging to this call from our stack using `pop()`. Remember from Step 4 that `pop()` does not erase the value from the underlying array—it only removes it from the active part of the stack.
- Use `info()` when entering and leaving `foo()` to observe how the stack changes during the recursive calls. Pay particular attention to the array elements that are no longer part of the stack after a call returns.

After all calls have returned, our stack is empty. Are the values `3`, `2`, and `1` still present in the underlying array? Compare this with the faded stack frames in the figure above. What does “removing” a value from the stack actually mean?

© Michael C. Lehn | CC BY-SA 4.0

Step 6 — Pointers and Pointers to Pointers

In the lecture videos, you saw that every variable occupies a location in memory and that each memory location has an address. A pointer is a variable that stores such an address.

Consider the following program (`ex1.abc`). Before running it, try to understand how the variables `a`, `b`, `p`, and `q` are related.

```
 1 @ <stdio.hdr>
 2
 3 fn main()
 4 {
 5     local
 6         a: int = 42,
 7         b: int = 123,
 8         p: ->int = &a,
 9         q: ->->int = &p;
10
11         printf("a = %d\n", a);
12         printf("b = %d\n", b);
13         printf("*p = %d\n", *p);
14         printf("**q = %d\n\n", **q);
15
16         // here some magic will happen ;-)
17
18         *p = 13;
19         printf("a = %d\n", a);
20         printf("b = %d\n\n", b);
21
22         **q = 31;
23         printf("a = %d\n", a);
24         printf("b = %d\n", b);
25 }
```

Task

1. Draw the memory layout of the variables `a`, `b`, `p`, and `q` when execution reaches line 10. For each variable, indicate the value stored in its memory location. For pointers, use arrows to show which memory location they refer to.

   Then predict how this memory layout changes when execution reaches lines 21 and 25.
2. Replace the comment in line 16

   ```
   // here some magic will happen ;-)
   ```

   with a single statement such that the assignments in lines 18 and 22 modify `b` instead of `a`.

   Before running the program, explain why your statement should have this effect. Then test your prediction.
3. Add `printf` statements after line 14 to investigate the actual memory layout. Use appropriate format specifiers to print
   A. the sizes of `a`, `b`, `p`, and `q`,
   B. the addresses of `a`, `b`, and `p`, and
   C. the values stored in `p` and `q`.
4. Compare the output from Task 3 with the memory layout you drew in Task 1. In particular, identify which addresses are stored in `p` and `q`, and explain what the expressions

   `*p`, `*q`, `**q`

   refer to.

© Michael C. Lehn | CC BY-SA 4.0

Step 7 — Type Casts: Interpreting Memory Differently

In Step 6, we used pointers to access values stored at particular memory addresses. But a memory address by itself does not tell us how the bytes stored there should be interpreted. The type of a pointer tells the compiler what kind of value we expect to find at that address.

In this step, we will deliberately reinterpret the same memory using a pointer of a different type.

Consider `ex2.abc`:

```
1 @ <stdio.hdr>
2
3 fn main()
4 {
5     local a: int = 0x12345678;
6
7     printf("a = 0x%x\n", a);
8
9     local ch: ->char = "ABCD";
10
11    printf("ch[0] = 0x%hhx\n", ch[0]);
12    printf("ch[1] = 0x%hhx\n", ch[1]);
13    printf("ch[2] = 0x%hhx\n", ch[2]);
14    printf("ch[3] = 0x%hhx\n", ch[3]);
15 }
```

Note

The format specifiers `%x` and `%hhx` are used here to print values in hexadecimal notation:
- `%x` prints an `int` value in hexadecimal.
- `%hhx` prints a `char` value in hexadecimal.

Task

1. **Exploring the memory layout**

   Before running the program, draw the memory layout of `a`, `ch`, `ch[0]`, `ch[1]`, `ch[2]`, and `ch[3]` when execution reaches line 10.

   Indicate which values are stored in the different memory locations and how ch is related to `ch[0]`.

2. **Array indexing and pointers**

   Rewrite lines 11–14 without using the characters `[` and `]`. The program should still produce exactly the same output.

3. **What is wrong with this pointer?**

   Change line 9 to

   ```
   9     local ch: ->char = &a;
   ```

   The compiler now complains.
   - Explain why this error is reasonable. What is the type of `&a`, and what type is expected for `ch`?
   - Suppose for a moment that the compiler accepted the program anyway. Draw how `a` and `ch` would then be related in memory. What would `ch` point to?

   Do not try to fix the error yet.

4. **Reinterpreting memory with a type cast**

   The compiler keeps track of the type of every expression. It therefore knows that `&a` has type `->int`, whereas `ch` is declared to have type `->char`.

   With an *explicit type cast*, we can tell the compiler to treat an expression as having a different type. Change line 9 to

   ```
   9     local ch: ->char = (->char)&a;
   ```

   The cast `(->char)` tells the compiler to treat the address `&a` as a pointer to `char`.
   Before running the program, predict what the four expressions `ch[0]`, `ch[1]`, `ch[2]`, and `ch[3]` will access.

© Michael C. Lehn | CC BY-SA 4.0

Then:

A. Compile and run the program. Determine values of `ch[0]`, `ch[1]`, `ch[2]`, `ch[3]`.
B. Use your observations to draw how the individual bytes of `a` are stored in memory.
C. Compare your result with the hexadecimal value `a = 0x12345678` In which order do the bytes `0x12`, `0x34`, `0x56`, and `0x78` appear in memory?
   The exact byte order depends on the target architecture. We are not trying to establish a universal ordering here—we are investigating how the machine on which you run the program represents the value.

5. **Looking at the machine code**
   The program `objdump` can be used to inspect the machine code of an executable. This will also be useful in the next task, so let us try it on the executable produced in this step.

   The output of `objdump` contains the bytes of the machine instructions in hexadecimal notation together with the corresponding assembly instructions. The exact output depends on the target architecture, compiler, and compiler options, but the basic idea is the same.

   If `ex2.abc` was compiled to `a.out`, run `objdump -d a.out`. Your output should be similar to:

```
…
0000000100000ed0 <_main>:
100000ed0: 48 83 ec 18                          subq    $24, %rsp
100000ed4: c7 44 24 0c 78 56 34 12              movl    $305419896, 12(%rsp)    ##
imm = 0x12345678
…
```

   Find the instruction in which the value `0x12345678` occurs. Compare the four bytes representing this value in the machine instruction with the byte layout of `a` that you observed in Task 4. *Do they appear in the same order?*

Step 8 — Addresses and Machine Instructions

In Step 7, we used an address to inspect the individual bytes stored in memory. But memory does not only contain data—it also contains the machine instructions of our program.

A compiler translates a program into a sequence of machine instructions. When the program is executed, these instructions are normally executed one after another. The machine instructions of a function are stored in memory, just like other bytes.

We can even obtain the address at which a function begins. Applying the address operator to a function name gives us the address of that function.

Consider the following program `ex3.abc`. It obtains the address of `main`, interprets this address as a pointer to `char`, and uses this pointer to inspect the first 20 bytes of `main`:

```
@ <stdio.hdr>

fn main()
{
    local ch: ->char = (->char)&main;

    printf("ch = %p\n", ch);
    for (local i: int = 0; i < 20; ++i) {
        printf("%hhx ", ch[i]);
    }
    printf("\n");
}
```

Task

Compile and run the program. Then use `objdump` as in Step 7 to inspect the machine instructions of the executable by running `objdump -d a.out`. Find the beginning of `main` in the `objdump` output and compare it with the output produced by the program. In particular, compare:

- the address printed for `ch` with the address of `main` shown by `objdump`, and
- the first 20 bytes printed by the program with the machine-code bytes shown for `main` by `objdump`.

Do they match?

© Michael C. Lehn | CC BY-SA 4.0

Step 9 — Function Pointers

In Step 8, we saw that a function has an address in memory. Just like other addresses, a function address can be stored in a pointer variable.

The type of such a pointer must describe the signature of the function it refers to. A function signature specifies the types of its parameters and its return type.

Consider the following program `ex4.abc`:

```
1 @ <stdio.hdr>
2
3 fn german(wert: int): int
4 {
5     printf("Hallo Welt! wert = %d\n", wert);
6     return wert + 42;
7 }
8
9 fn english(value: int): int
10 {
11     printf("hello, world! value = %d\n", value);
12     return value + 123;
13 }
14
15 fn main()
16 {
17     local msg: ->fn(val: int): int;
18
19     msg = &german;
20
21     local foo: int = (*msg)(12);
22
23     printf("foo = %d\n", foo);
24 }
```

Task

1. Compile and run the program. Make sure you understand its output. In particular, identify which function is called by `(*msg)(12)` and why.
2. Modify the program so that after the assignment to `msg`, it points to `english` instead of `german`. Run the program again and explain the changed output.
3. The name of a parameter is not part of a function signature. Only its type matters. Therefore, the name `val` in `local msg: ->fn(val: int): int` is not required. Verify that the declaration can be changed to

   ```
   local msg: ->fn(: int): int;
   ```

   without changing the behavior of the program. What information about a function must therefore be known in order to call it through a function pointer?
4. Functions can, of course, also return pointers—and these pointers can themselves be pointers to functions.
   Add the following function to your program:

   ```
   fn selectLang(sel: int): ->fn(: int): int
   {
       if (sel == 0) {
           return &german;
       } else {
           return &english;
       }
   }
   ```

   Use `selectLang` to initialize `msg`, for example so that

   ```
   msg = selectLang(0);
   ```

   makes `msg` point to `German`.

© Michael C. Lehn | CC BY-SA 4.0

5. The function `scanf`, declared in `stdio.hdr`, can be used to read a value into a variable. Consider the following small example:

```
@ <stdio.hdr>

fn main()
{
    local a: int;

    printf("a = ");
    scanf("%d", &a);
    printf("a = %d\n", a);
}
```

   Compile and run the program.

   Why must we pass the address of `a` to `scanf` rather than the value of a itself? Relate your explanation to what you learned about pointers in the previous steps.

6. Finally, combine these ideas. Modify the program so that it first reads two integer values:
   - The first value selects whether german or english is called.
   - The second value is passed as the argument to the selected function.

   Use selectLang and a function pointer to perform the call.

   For example, your program could behave like this:

```
MCL:tmp lehn$ ./a.out
lang = 1
val = 34
hello, world! value = 34
result = 157
MCL:tmp lehn$ ./a.out
lang = 0
val = 123
Hallo Welt! wert = 123
result = 165
```

© Michael C. Lehn | CC BY-SA 4.0